

# DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI

## Chương 4 - Huấn luyện Mô hình (Training Models)

TS. Trần Thành Thắng

Đại học Đông Á

August 11, 2026

# Nội dung Chương trình

1. Hồi quy tuyến tính & Phương trình chuẩn tắc
2. Hạ Gradient (Gradient Descent)
3. Hồi quy đa thức, Đường cong học tập & Chính quy hóa
4. Hồi quy Logistic & Softmax

- Hiểu rõ cách thức hoạt động bên trong của các mô hình học máy thay vì coi chúng là "hộp đen".
- Tìm hiểu và áp dụng các phương pháp giải Hồi quy tuyến tính (Phương trình chuẩn tắc, SVD, Gradient Descent).
- Phân biệt được các loại Gradient Descent (Batch, Stochastic, Mini-batch) và ảnh hưởng của Tốc độ học (Learning Rate).
- Hiểu cách xử lý dữ liệu phi tuyến tính bằng Hồi quy đa thức.
- Phát hiện Quá khớp (Overfitting) thông qua Đường cong học tập và các kỹ thuật Chính quy hóa (Ridge, Lasso, Elastic Net, Early Stopping).
- Tìm hiểu Hồi quy Logistic cho phân loại nhị phân và Hồi quy Softmax cho đa lớp.

# 1. Hồi quy tuyến tính & Phương trình chuẩn tắc

# Hồi quy tuyến tính (Linear Regression)

- Hồi quy tuyến tính là một trong những mô hình đơn giản nhất. Mô hình sẽ đưa ra dự đoán bằng cách tính tổng trọng số của các đặc trưng đầu vào, cộng với một hằng số gọi là số hạng độ lệch (bias term) hoặc hệ số chặn (intercept term).
- Mô hình dự đoán hồi quy tuyến tính (Dạng toán học):**

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \quad (1)$$

- Dạng Vector (Rút gọn):**

$$\hat{y} = h_{\theta}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x} \quad (2)$$

- Trong đó:  $\boldsymbol{\theta}$  là vector tham số mô hình,  $\mathbf{x}$  là vector đặc trưng của trường hợp.

# Hàm chi phí MSE (Mean Squared Error)

- Để huấn luyện mô hình Hồi quy tuyến tính, chúng ta cần tìm giá trị cho vector tham số  $\theta$  nhằm tối thiểu hóa sai số giữa giá trị dự đoán và giá trị thực tế.
- Thước đo phổ biến nhất là Hàm chi phí Lỗi Bình phương Trung bình (MSE - Mean Squared Error).
- **Công thức MSE cho mô hình hồi quy tuyến tính:**

$$MSE(\mathbf{X}, h_{\theta}) = \frac{1}{m} \sum_{i=1}^m \left( \theta^{\top} \mathbf{x}^{(i)} - y^{(i)} \right)^2 \quad (3)$$

- Mục tiêu của quá trình huấn luyện là tìm  $\theta$  để MSE đạt giá trị nhỏ nhất.

# Phương trình chuẩn tắc (Normal Equation)

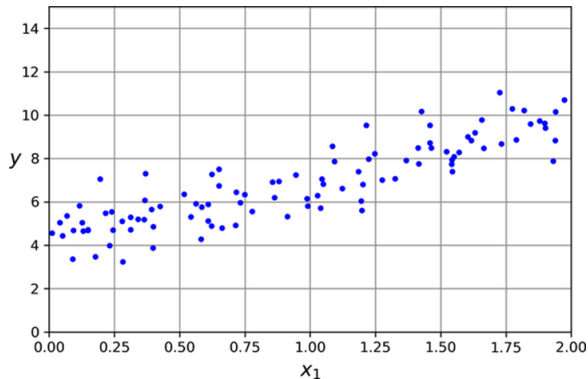
- Để tìm giá trị  $\theta$  làm tối thiểu hóa hàm chi phí, có một nghiệm dạng đóng (closed-form solution) – một phương trình toán học cho ra kết quả trực tiếp. Nó được gọi là Phương trình chuẩn tắc.
- **Công thức Phương trình chuẩn tắc:**

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (4)$$

- $\hat{\theta}$  là giá trị  $\theta$  làm giảm thiểu hàm chi phí.
- $\mathbf{y}$  là vector của các giá trị mục tiêu (từ  $y^{(1)}$  đến  $y^{(m)}$ ).

# Minh họa: Tập dữ liệu tuyến tính ngẫu nhiên

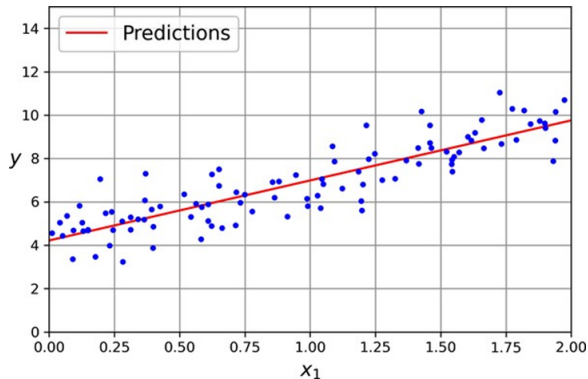
- Để kiểm tra Phương trình chuẩn tắc, chúng ta có thể tạo một số dữ liệu nhìn có vẻ tuyến tính một cách ngẫu nhiên.
- Công thức sử dụng để tạo dữ liệu:  
 $y = 4 + 3x + \text{nhiều Gauss.}$
- Biểu đồ bên cạnh cho thấy các điểm dữ liệu được tạo ra. Dữ liệu này sẽ được dùng để khớp mô hình.



Hình 4-1. Dữ liệu tuyến tính được tạo ngẫu nhiên

# Minh họa: Dự đoán của Hồi quy tuyến tính

- Sau khi tính toán được  $\hat{\theta}$  từ Phương trình chuẩn tắc, chúng ta có thể vẽ đường thẳng dự đoán.
- Hình bên cạnh cho thấy các dự đoán của mô hình (đường thẳng màu đỏ) khớp với tập dữ liệu ngẫu nhiên của chúng ta như thế nào.
- Đường thẳng này đi qua trung tâm của các điểm dữ liệu, cho thấy mô hình đã học được xu hướng chung của dữ liệu.



Hình 4-2. Dự đoán của mô hình hồi quy tuyến tính

# Thực hiện với Scikit-Learn và Singular Value Decomposition

- Trong thực tế, Scikit-Learn sử dụng **Singular Value Decomposition (SVD)** thay vì tính trực tiếp Phương trình chuẩn tắc.
- **Lý do:** Ma trận  $\mathbf{X}^T \mathbf{X}$  có thể không khả nghịch (chẳng hạn khi số lượng đặc trưng lớn hơn số lượng trường hợp huấn luyện). Phương pháp SVD luôn xử lý được các trường hợp này bằng ma trận giả nghịch đảo (pseudoinverse).
- Đoạn mã Scikit-Learn rất đơn giản:  

```
lin_reg = LinearRegression()  
lin_reg.fit(X, y)
```

# Độ phức tạp tính toán

- Cả phương trình chuẩn tắc và cách tiếp cận SVD đều trở nên **rất chậm** khi số lượng đặc trưng tăng lên đáng kể.
- Độ phức tạp tính toán của việc đảo ngược ma trận (như Phương trình chuẩn tắc) là khoảng  $\mathcal{O}(n^{2.4})$  đến  $\mathcal{O}(n^3)$ .
- Phương pháp SVD được Scikit-Learn sử dụng là khoảng  $\mathcal{O}(n^2)$ . Tức là, nếu tăng gấp đôi số lượng đặc trưng, thời gian tính toán tăng gấp 4 lần.
- **Ưu điểm:** Cả hai phương pháp này lại là tuyến tính đối với số lượng trường hợp huấn luyện (độ phức tạp  $\mathcal{O}(m)$ ). Chúng xử lý rất tốt tập huấn luyện lớn miễn là nó vừa với bộ nhớ.

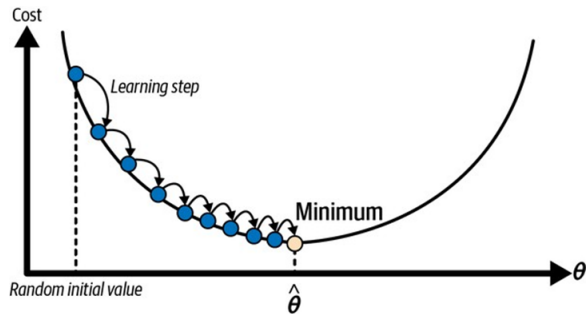
## 2. Hạ Gradient (Gradient Descent)

# Gradient Descent là gì?

- **Gradient Descent (Hạ Gradient)** là một thuật toán tối ưu hóa chung có khả năng tìm ra giải pháp tối ưu cho nhiều loại vấn đề.
- Ý tưởng chung: Tinh chỉnh các tham số theo vòng lặp để giảm thiểu hàm chi phí.
- Ví dụ trực quan: Tưởng tượng bạn bị lạc trên núi trong một màn sương mù dày đặc, và bạn chỉ có thể cảm nhận độ dốc của mặt đất dưới chân mình. Chiến lược tốt nhất để xuống đáy thung lũng là bước xuống dốc theo hướng dốc nhất.
- Nó đo lường gradient (đạo hàm) cục bộ của hàm lỗi so với vector tham số  $\theta$ , và đi theo hướng giảm của gradient cho đến khi gradient bằng 0.

# Minh họa: Nguyên lý hoạt động của Gradient Descent

- Kích thước bước nhảy được xác định bởi siêu tham số **tốc độ học (learning rate)**.
- Bắt đầu với một giá trị  $\theta$  ngẫu nhiên, thuật toán dần dần tiến gần đến điểm cực tiểu.
- Hình bên minh họa sự mô phỏng quá trình này: các bước đi dần hội tụ tại cực tiểu toàn cục.



Hình 4-3. Gradient Descent

# Phân tích ảnh hưởng của Tốc độ học (Learning Rate)

- Tốc độ học (Learning Rate -  $\eta$ ) là một siêu tham số quan trọng nhất của thuật toán Gradient Descent.
- **Tốc độ học quá nhỏ:**
  - Thuật toán sẽ phải trải qua rất nhiều vòng lặp để hội tụ.
  - Mất rất nhiều thời gian.
- **Tốc độ học quá cao:**
  - Bạn có thể "nhảy cóc" qua thung lũng và kết thúc ở bên kia, thậm chí cao hơn trước.
  - Làm cho thuật toán phân kỳ (diverge), giá trị hàm chi phí ngày càng tăng và không bao giờ tìm được điểm tối ưu.